

C DECISION MAKING

THE IF-ELSE CONTROL STRUCTURES

Comprehensive 12-Page Logical Guide

01. Decision Making in C

In programming, decision-making structures require that the programmer specify one or more conditions to be evaluated by the program.

Concept: Decision-making statements allow a program to skip or execute certain blocks of code based on whether a condition is **True** or **False**.

The `if-else` statement is the most common way to implement this logic in C.

02. The Single 'if' Statement

The `if` statement is used to execute a block of code **only** if a specified condition is true.

```
if (test_condition) {  
    // code to execute if true  
}
```

If the condition evaluates to 0 (False), the compiler simply skips the code inside the curly braces and moves to the next statement.

03. The 'if-else' Duo

When you have two mutually exclusive paths, you use `if-else`. If the condition is true, the **if** block executes; otherwise, the **else** block runs.

```
if (age >= 18) {  
    printf("You can vote.");  
} else {  
    printf("Too young!");  
}
```

Only one of these two blocks will ever run during a single execution.

04. Visualizing Logic

[Start] → [Condition?]

└─ (Yes) → [IF Block] → [End]

└─ (No) → [ELSE Block] → [End]

This "branching" allows programs to handle different inputs gracefully without crashing or following a linear, rigid path.

05. Multi-way Decisions

When there are more than two possibilities, you use the `else if` ladder. This allows you to check multiple conditions in sequence.

```
if (marks > 90) { ... }  
else if (marks > 70) { ... }  
else if (marks > 40) { ... }  
else { ... }
```

The program checks conditions from top to bottom. As soon as one is true, it executes that block and ignores the rest of the ladder.

06. Nested Structures

You can place an `if` statement inside another `if` statement. This is known as **Nesting**.

```
if (is_member) {  
    if (has_coupon) {  
        price = 10;  
    }  
}
```

This is useful for complex scenarios where one condition depends on another being passed first.

07. Conditions & Relational Ops

Conditions in `if` statements often use Relational Operators:

- `==` (Equal)
- `!=` (Not Equal)
- `&&` (Logical AND)
- `||` (Logical OR)

Note: `if(1)` is always true, `if(0)` is always false.

08. Syntax Shortcuts

If the block inside an `if` or `else` contains only **one** line of code, the curly braces `{ }` are optional.

```
if (x > y)
    max = x;
else
    max = y;
```

Warning: Industrially, it is better to always use braces to prevent bugs during future code updates.

09. If-Else vs Switch

Feature	if-else	switch
Flexibility	Checks ranges (>, <)	Checks fixed values
Types	Works with all types	Works with int/char
Clarity	Better for logical flow	Better for menu lists

10. Troubleshooting Pitfalls

- **The Assignment Error:** Writing `if(x = 5)` instead of `if(x == 5)`.
- **Dangling Else:** Confusing which `if` an `else` belongs to in nested structures.
- **Logic Gaps:** Forgetting the final `else` to catch unhandled cases.

11. Critical Summary

- Use **if** for single checks.
- Use **if-else** for binary choices.
- Use **else-if** for multiple choice lists.
- Remember: 0 is false, everything else is true.

END OF IF-ELSE GUIDE